

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN
CS202 - PROGRAMING SYSTEM

Final Project

Đề tài: Game Development

Môn học: Data Structure

Sinh viên thực hiện:

Đoàn Tuấn Anh (24125002)

Nguyễn Trọng Văn Viết(24125023)



Mục lục

1	Gameplay Overview	1
1.1	Core Adventure Features	1
1.1.1	Dynamic Combat System	1
1.1.2	Strategic Inventory Management	1
1.1.3	Rescue and Recruitment System	1
1.1.4	Exploration and Discovery	2
2	Complete Codebase Hierarchy	2
3	Design Pattern Implementations	3
3.1	State Pattern - Game Screen Management	3
3.2	Singleton Pattern - Resource Management	4
3.3	Strategy Pattern - Status Effects	6
3.4	Component Pattern - Entity Attributes	6
3.5	Composite Pattern - Entity Hierarchy	8
4	Entity System Hierarchy	9
5	State Management Hierarchy	9
6	Resource Management Hierarchy	10
7	Animation System Hierarchy	10
8	Combat System Hierarchy	11
9	World System Hierarchy	12
10	Architectural Analysis	14
10.1	Hierarchy Strengths	14
10.2	Key Design Decisions	15

1 Gameplay Overview

Embark on an Epic Adventure in the Mystical Jungle Realm!

Welcome to an immersive 2D action-adventure where danger lurks behind every tree and treasure awaits the brave! Take control of valiant heroes, including the mighty Knight, as you venture into hostile territories filled with enemies, ancient treasures, and imprisoned allies waiting for rescue.

1.1 Core Adventure Features

1.1.1 Dynamic Combat System

- Engage in thrilling real-time combat against various enemy types using an advanced weapon system
- Master different weapon classes with unique attack patterns and strategic advantages
- Experience satisfying knockback physics and visual combat feedback
- Utilize a sophisticated projectile system for ranged encounters

1.1.2 Strategic Inventory Management

- Discover and collect powerful weapons scattered throughout the world
- Carry multiple weapons and switch between them instantly during combat (E key)
- Make tactical decisions about which weapons to keep or discard (R key)
- Scavenge the battlefield for better equipment to enhance your combat effectiveness

1.1.3 Rescue and Recruitment System

- Locate imprisoned allies trapped in enemy strongholds and dangerous areas
- Risk everything to free captured friends who will join your cause

- Build a growing party of loyal companions, each with unique abilities
- Transform solo adventures into epic group expeditions as your rescued allies fight alongside you

1.1.4 Exploration and Discovery

- Navigate through beautifully crafted tile-based environments in the mysterious Jungle realm
- Interact with the world using intuitive controls - approach and investigate objects (F key)
- Uncover hidden treasures, secret passages, and valuable resources
- Experience dynamic world layers with atmospheric backgrounds and interactive elements

2 Complete Codebase Hierarchy

```
v:\Game\  
include\  
    Book\           // Core game engine components  
    Control\        // State and resource management  
    Object\         // Game objects (weapons, characters, etc.)  
    Screens\        // Game state implementations  
SourceCode\  
    Game\           // Core implementations  
    Screens\        // State implementations  
    Control\        // Management implementations  
build\             // Build artifacts
```

3 Design Pattern Implementations

3.1 State Pattern - Game Screen Management

Location: Control/State.hpp

```
// filepath: v:\Game\include\Control\State.hpp
#pragma once
#include <memory>
#include <SFML/System/Time.hpp>
#include <SFML/Window/Event.hpp>

class StateStack;

class State
{
public:
    typedef std::unique_ptr<State> Ptr;

    struct Context
    {
        sf::RenderWindow* window;
        // Additional context data like resources, camera, etc.
    };

    State(StateStack& stack, Context context);
    virtual ~State();

    // Template Method Pattern - defines the interface all states must
    // follow
    virtual void draw() = 0;
    virtual bool update(sf::Time dt) = 0;
    virtual bool handleEvent(const sf::Event& event) = 0;

protected:
    // Command Pattern - state change requests
```

```
void requestStackPush(States::ID stateID);  
void requestStackPop();  
void requestStackClear();  
Context getContext() const;  
  
private:  
    StateStack* mStack;  
    Context mContext;  
};
```

Listing 1: State Pattern Interface Implementation

Usage in Title State:

```
// filepath: v:\Game\SourceCode\Screens\Title.cpp  
bool Title::handleEvent(const sf::Event& event)  
{  
    if (event.type == sf::Event::KeyPressed)  
    {  
        if(event.key.code == sf::Keyboard::Enter)  
        {  
            requestStackPop();  
            requestStackPush(States::Lobby); // State transition  
        }  
    }  
    return false;  
}
```

Listing 2: State Pattern Usage Example

3.2 Singleton Pattern - Resource Management

Location: Control/ResourceManager.hpp

```
// filepath: v:\Game\include\Control\ResourceManager.hpp  
class ResourceManager  
{  
public:
```

```
// Singleton access point
static ResourceManager& getInstance()
{
    static ResourceManager instance;
    return instance;
}

template<typename Resource>
Resource& get(typename Resource::ID id);

template<typename Resource>
void load(typename Resource::ID id, const std::string& filename);

// Prevent copying (Singleton enforcement)
ResourceManager(const ResourceManager&) = delete;
ResourceManager& operator=(const ResourceManager&) = delete;

private:
    ResourceManager() = default;
    ~ResourceManager() = default;
};
```

Listing 3: Singleton Pattern Implementation

Usage in Title State:

```
// filepath: v:\Game\SourceCode\Screens\Title.cpp
Title::Title(StateStack& stack, Context context)
    : State(stack, context)
{
    map = new TileMap(ResourceManager::getInstance().get<TileMap>(Map::ID
        ::Title));
}
```

Listing 4: Singleton Pattern Usage

3.3 Strategy Pattern - Status Effects

Location: Book/StrategyClass.hpp

```
// filepath: v:\Game\include\Book\StrategyClass.hpp
class IStatusEffect
{
public:
    virtual ~IStatusEffect() = default;
    virtual void apply(Character* character) = 0;
    virtual bool update(sf::Time dt) = 0;
    virtual bool isExpired() const = 0;
    virtual std::string getName() const = 0;
};
```

Listing 5: Strategy Pattern Interface

Usage in Character Class:

```
// filepath: v:\Game\include\Book\Character.hpp
class Character : public Entity
{
protected:
    std::vector<std::shared_ptr<IStatusEffect>> statusEffect;

public:
    // Strategy pattern allows different status effects to be applied
    void applyStatusEffect(std::shared_ptr<IStatusEffect> effect);
    void updateStatusEffects(sf::Time dt);
};
```

Listing 6: Strategy Pattern Usage in Character

3.4 Component Pattern - Entity Attributes

Location: Book/EntityComponent.hpp

```
// filepath: v:\Game\include\Book\EntityComponent.hpp
class EntityAttributeActionComponent
```

```
{
private:
    std::map<CharacterResourceType, std::pair<float, float>> attributes;
    // current, max

public:
    EntityAttributeActionComponent();

    void setAttribute(CharacterResourceType type, float current, float max
        );
    float getAttribute(CharacterResourceType type) const;
    float getMaxAttribute(CharacterResourceType type) const;
    void modifyAttribute(CharacterResourceType type, float amount);

    void update(sf::Time dt);
};
```

Listing 7: Component Pattern Implementation

Usage in Character:

```
// filepath: v:\Game\include\Book\Character.hpp
class Character : public Entity
{
protected:
    EntityAttributeActionComponent attributes; // Composition over
    inheritance

public:
    void setAttribute(CharacterResourceType type, float current, float max
        );
    virtual int getHealth();
    virtual int getMana();
    virtual int getShield();
};
```

Listing 8: Component Pattern Usage

3.5 Composite Pattern - Entity Hierarchy

Location: Book/Entity.hpp

```
// filepath: v:\Game\include\Book\Entity.hpp
class Entity : public sf::Drawable, public std::enable_shared_from_this<
    Entity>
{
public:
    typedef std::shared_ptr<Entity> Ptr;

protected:
    std::vector<Ptr> children; // Composite pattern - can contain other
                             // entities
    Entity* parent;

public:
    // Composite operations
    void attachChild(Ptr child);

    // Operations cascade to children
    virtual bool update(sf::Time dt)
    {
        // Update children entities
        for (Ptr& child : children)
        {
            child->update(dt);
        }
        return false;
    }
};
```

Listing 9: Composite Pattern Implementation

4 Entity System Hierarchy

Complete Entity Class Hierarchy

```
sf::Drawable
├── Entity (Base class for all game objects)
│   ├── Character (Player-controllable entities)
│   │   └── Knight (Specific character implementation)
│   ├── Projectile2 (Modern projectile system)
│   ├── Weapon2 (Modern weapon system)
│   ├── Chest (Interactive storage containers)
│   └── Environmental Objects
└── TileMap (World rendering system)
    ├── Layer (Abstract layer interface)
    │   ├── TileLayer (Static tile rendering)
    │   ├── ObjectLayer (Entity management)
    │   └── ImageLayer (Background images)
```

5 State Management Hierarchy

State Management System

```
StateStack (State container and manager)
├── State (Abstract base state)
│   ├── Title (Main menu state)
│   ├── Lobby (Character selection/preparation)
│   └── Jungle (Gameplay state)
```

State Context System

```

Context (Shared state data)
├─ sf::RenderWindow* window
└─ Additional Context Data...

```

6 Resource Management Hierarchy

Resource Management System

```

ResourceManager (Singleton)
├─ ResourceHolder<Resource
│   ├── Textures
│   ├── Maps (TileMap objects)
│   └─ Other Resource Types...

```

Resource Types

```

Resource Types
├─ TileMap (Level data and structure)
├─ Textures (Character sprites
=0 └─ Additional Resources...

```

7 Animation System Hierarchy

Animation System

```

MovingAnimation (Base animation class)
├─ Character_MovingAnimation (Character-specific animation)

```

Animation Integration

Character

- └─ `std::unique_ptr<MovingAnimation> movingAnimation`
- └─ Animation responds to character movement and actions

8 Combat System Hierarchy

Combat System

Combat System

- └─ Weapon System
 - └─ Weapon2 (Base weapon class)
 - └─ WeaponLoader (Factory for weapon creation)
- └─ Projectile System
 - └─ Projectile2 (Base projectile class)
- └─ Status Effects
 - └─ IStatusEffect (Strategy interface)
- └─ Inventory System
 - └─ Inventory (Weapon container)

Location: Book/Inventory.hpp

```
// filepath: v:\Game\include\Book\Inventory.hpp
class Inventory
{
private:
    std::vector<std::shared_ptr<Weapon2>> weapons;
    std::shared_ptr<Weapon2> currentWeapon;
    int currentWeaponIndex;

public:
    // Weapon management
```

```

void SwitchWeapon();           // E key functionality
void removeCurrentWeapon();    // R key functionality

// Rendering
void draw(sf::RenderTarget& target, sf::RenderStates states) const;
};

```

Listing 10: Inventory System Implementation

9 World System Hierarchy

World System

```

TileMap (Main world container)
├─ Layer System
│   ├── TileLayer (Static tile rendering)
│   ├── ObjectLayer (Entity management)
│   └── ImageLayer (Background images)
├─ Entity Management
│   ├── std::vector<Entity*> entities
│   └── Entity* GetClosestItem(sf::Vector2f position)
└─ Update/Render System
    ├── bool update(sf::Time dt)
    ├── void draw(...)
    └── bool handleEvent(...)

```

Location: Book/TileMap.hpp

```

// Layer Base Class
class Layer
{
public:

```

```
bool visible = true;
virtual ~Layer() = default;

virtual void draw(sf::RenderTarget& target, sf::RenderStates states)
    const = 0;
virtual void setPosition(float x, float y) = 0;
};
```

Listing 11: Layer Base Class Implementation

```
// TileLayer - Static world tiles
class TileLayer : public Layer
{
public:
    std::vector<int> data;
    std::vector<sf::VertexArray> vertices;
    std::vector<Tileset> tilesets;
    sf::Vector2f position;
    int width;
    int height;
};
```

Listing 12: TileLayer Implementation

```
// ObjectLayer - Contains game entities
class ObjectLayer : public Layer
{
public:
    std::vector<Entity*> entities;
    sf::Vector2f position;

    virtual void draw(sf::RenderTarget& target, sf::RenderStates states)
        const override;
};
```

Listing 13: ObjectLayer Implementation

```
// ImageLayer - Background images
```

```
class ImageLayer : public Layer
{
public:
    std::string File;
    sf::Texture texture;
    sf::Vector2f position;
    sf::Sprite sprite;

    virtual void draw(sf::RenderTarget& target, sf::RenderStates states)
        const override;
};
```

Listing 14: ImageLayer Implementation

10 Architectural Analysis

10.1 Hierarchy Strengths

1. Clear Separation of Concerns

- Entity System: Game object behavior
- State System: Game flow management
- Resource System: Asset management
- Animation System: Visual representation

2. Design Pattern Integration

- State Pattern: Clean game screen management
- Singleton Pattern: Global resource access
- Strategy Pattern: Flexible status effects
- Component Pattern: Modular entity features
- Composite Pattern: Entity parent-child relationships

3. Extensibility

- Virtual interfaces allow easy extension
- Template-based resource system supports new types
- Component system enables feature mixing

4. Performance Considerations

- Smart pointer usage for automatic memory management
- Entity pooling through vector containers
- Efficient resource sharing via singleton

10.2 Key Design Decisions

1. **Entity-Component Hybrid:** Uses inheritance for core entity types with component composition for features
2. **State-Based Architecture:** Clear separation of game phases (Title, Lobby, Jungle)
3. **Resource Centralization:** Single point of access for all game assets
4. **Event Propagation:** Events cascade through entity hierarchy